Andy Garcia
Assignment 4
EECS 348
10/13

- Two GenAI programs used: ChatGPT and Google Gemini
- Accessed through browser
- Prompt:

Write a C++ program in a single file that solves Sudoku puzzles using recursive depth-first search with backtracking. Use an object-oriented design with classes, not standalone functions. The program must read five files named puzzle1.txt through puzzle5.txt, where each file is a 9x9 grid of digits 1-9 separated by spaces, with an underscore (_) for blank cells. For each file, print the file name, the original puzzle, and then every solution found, or “No solution found” if none exists. The puzzle may have multiple solutions, so find and print all of them. Add a comment to every line of code.

ChatGPT’s Code:

```
#include <iostream>  // Provides input and output operations.
#include <fstream>   // Provides file input operations.
#include <string>    // Provides the string class.
#include <vector>    // Provides the vector container.
using namespace std; // Allows standard library names without std::.

class Sudoku { // Defines a class that represents and solves a Sudoku puzzle.
private: // Makes the following members accessible only inside the class.
   int grid[9][9]; // Stores the Sudoku grid, using 0 for blank cells.
   int solutionCount; // Tracks the number of solutions found.

   bool isValid(int row, int col, int num) { // Checks whether num can be placed in the specified cell.
      for (int i = 0; i < 9; i++) { // Examines every row and column position.
         if (grid[row][i] == num) return false; // Rejects a number already present in the row.
         if (grid[i][col] == num) return false; // Rejects a number already present in the column.
      } // Ends the row and column checks.
      int startRow = (row / 3) * 3; // Calculates the first row of the cell's 3x3 box.
      int startCol = (col / 3) * 3; // Calculates the first column of the cell's 3x3 box.
      for (int r = startRow; r < startRow + 3; r++) { // Iterates through the three rows of the box.
         for (int c = startCol; c < startCol + 3; c++) { // Iterates through the three columns of the box.
            if (grid[r][c] == num) return false; // Rejects a number already present in the 3x3 box.
         } // Ends the inner box loop.
      } // Ends the outer box loop.
```

```
        return true; // Confirms that the number is valid for this cell.
    } // Ends the isValid method.

    void printGrid() { // Prints the current Sudoku grid.
        for (int r = 0; r < 9; r++) { // Visits each row.
            for (int c = 0; c < 9; c++) { // Visits each column in the current row.
                cout << grid[r][c] << " "; // Prints the value stored in the current cell.
            } // Ends the column loop.
            cout << '\n'; // Moves to the next output line after each row.
        } // Ends the row loop.
        cout << '\n'; // Adds a blank line after the grid.
    } // Ends the printGrid method.

    bool findEmptyCell(int &row, int &col) { // Finds the next blank cell and returns its position by
reference.
        for (row = 0; row < 9; row++) { // Searches rows from top to bottom.
            for (col = 0; col < 9; col++) { // Searches columns from left to right.
                if (grid[row][col] == 0) return true; // Returns true when a blank cell is found.
            } // Ends the column search.
        } // Ends the row search.
        return false; // Returns false when the grid has no blank cells.
    } // Ends the findEmptyCell method.

    void solveRecursive() { // Uses recursive depth-first search and backtracking to find every
solution.
        int row = 0; // Initializes the row index for the next blank cell.
        int col = 0; // Initializes the column index for the next blank cell.
        if (!findEmptyCell(row, col)) { // Checks whether every cell has been filled.
            solutionCount++; // Records the newly discovered complete solution.
            cout << "Solution " << solutionCount << ":\n"; // Prints the solution number.
            printGrid(); // Prints the completed Sudoku grid.
            return; // Returns to the previous recursive call to continue searching.
        } // Ends the completed-grid check.
        for (int num = 1; num <= 9; num++) { // Tries every possible digit for the blank cell.
            if (isValid(row, col, num)) { // Checks whether the current digit follows Sudoku rules.
                grid[row][col] = num; // Places the candidate digit into the blank cell.
                solveRecursive(); // Recursively searches for a complete solution.
                grid[row][col] = 0; // Backtracks by clearing the cell to try another candidate.
            } // Ends the validity check.
        } // Ends the candidate-digit loop.
    } // Ends the recursive solving method.

public: // Makes the following methods accessible outside the class.
    Sudoku() { // Initializes a new Sudoku object.
```

```
        solutionCount = 0; // Starts the solution counter at zero.
        for (int r = 0; r < 9; r++) { // Visits every row of the grid.
            for (int c = 0; c < 9; c++) { // Visits every column of the grid.
                grid[r][c] = 0; // Initializes every cell as blank.
            } // Ends the column initialization loop.
        } // Ends the row initialization loop.
    } // Ends the constructor.

    bool loadFromFile(const string &filename) { // Loads a puzzle from the specified text file.
        ifstream file(filename); // Opens the puzzle file for reading.
        if (!file.is_open()) return false; // Reports failure if the file cannot be opened.
        for (int r = 0; r < 9; r++) { // Reads nine rows from the file.
            for (int c = 0; c < 9; c++) { // Reads nine cells from each row.
                string token; // Stores the next cell's text representation.
                if (!(file >> token)) return false; // Rejects a file with missing cell values.
                if (token == "_") { // Checks whether the cell represents a blank.
                    grid[r][c] = 0; // Stores zero to represent a blank cell.
                } else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9') { // Validates a digit
from 1 through 9.
                    grid[r][c] = token[0] - '0'; // Converts the character digit into an integer.
                } else { // Handles invalid cell contents.
                    return false; // Rejects any cell that is neither a digit nor an underscore.
                } // Ends the cell-token validation.
            } // Ends the column-reading loop.
        } // Ends the row-reading loop.
        file.close(); // Closes the input file after reading the puzzle.
        return true; // Reports that the puzzle was loaded successfully.
    } // Ends the loadFromFile method.

    void printOriginal() { // Prints the puzzle before solving it.
        cout << "Original puzzle:\n"; // Labels the original grid.
        printGrid(); // Displays the original puzzle grid.
    } // Ends the printOriginal method.

    void solve() { // Starts the search for all possible solutions.
        solutionCount = 0; // Resets the solution counter before searching.
        solveRecursive(); // Starts recursive depth-first search.
        if (solutionCount == 0) { // Checks whether the search found any solutions.
            cout << "No solution found\n"; // Reports that no complete solution exists.
        } // Ends the no-solution check.
        cout << "Total solutions found: " << solutionCount << "\n\n"; // Prints the total number of
solutions.
    } // Ends the solve method.
}; // Ends the Sudoku class definition.
```

```
int main() { // Defines the program entry point.
   for (int i = 1; i <= 5; i++) { // Processes puzzle files numbered from 1 through 5.
      string filename = "puzzle" + to_string(i) + ".txt"; // Constructs the current puzzle filename.
      cout << "========================================\n"; // Prints a separator
between puzzles.
      cout << "File: " << filename << "\n\n"; // Prints the name of the file being processed.
      Sudoku puzzle; // Creates a Sudoku object for the current puzzle.
      if (!puzzle.loadFromFile(filename)) { // Attempts to load the current puzzle file.
         cout << "Error: Could not load a valid puzzle from " << filename << "\n\n"; // Reports a
missing or invalid input file.
         continue; // Skips to the next puzzle file.
      } // Ends the file-loading check.
      puzzle.printOriginal(); // Prints the original puzzle before searching.
      puzzle.solve(); // Finds and prints every solution for this puzzle.
   } // Ends the loop over all five files.
   return 0; // Indicates that the program completed successfully.
} // Ends the main function.
```

Gemini's Code:

```
#include <iostream>  // Include input-output stream library for console printing
#include <fstream>   // Include file stream library for reading puzzle text files
#include <vector>    // Include vector library to store the 9x9 grid and list of solutions
#include <string>    // Include string library for handling file names and strings

class SudokuSolver {   // Define a class to represent and solve the Sudoku puzzle
private:               // Private section to encapsulate board data and helper methods
   std::vector<std::vector<int>> board; // 2D vector representing the 9x9 Sudoku board

   bool isValid(int row, int col, int num) { // Helper method to check if a number can be placed at
a specific cell
      for (int i = 0; i < 9; ++i) {         // Loop through each column in the current row
         if (board[row][i] == num) return false; // Return false if the number already exists in the
row
      }
      for (int i = 0; i < 9; ++i) {         // Loop through each row in the current column
         if (board[i][col] == num) return false; // Return false if the number already exists in the
column
      }
      int startRow = row - row % 3;         // Calculate the starting row index of the 3x3 subgrid
      int startCol = col - col % 3;         // Calculate the starting column index of the 3x3 subgrid
      for (int r = 0; r < 3; ++r) {         // Loop through the 3 rows of the subgrid
```

```
            for (int c = 0; c < 3; ++c) {     // Loop through the 3 columns of the subgrid
                if (board[startRow + r][startCol + c] == num) return false; // Return false if found in
subgrid
            }
        }
        return true;                        // Return true if the placement is valid across row, col, and
subgrid
    }

    void solveRec(int row, int col, std::vector<std::vector<std::vector<int>>>& solutions) { //
Recursive backtracking method
        if (row == 9) {                     // Check if we have successfully processed past the last row
            solutions.push_back(board);       // Store a copy of the current valid board as a solution
            return;                         // Return from this recursive branch to search for more solutions
        }
        int nextRow = (col == 8) ? row + 1 : row; // Determine the row index for the next recursive
step
        int nextCol = (col == 8) ? 0 : col + 1; // Determine the column index for the next recursive
step

        if (board[row][col] != 0) {           // Check if the current cell is already filled from the initial
puzzle
            solveRec(nextRow, nextCol, solutions); // Recurse to the next cell without changing
current cell
        } else {                            // If the current cell is blank (represented by 0)
            for (int num = 1; num <= 9; ++num) { // Try digits 1 through 9 sequentially
                if (isValid(row, col, num)) { // Check if placing the current digit is valid
                    board[row][col] = num;    // Place the number temporarily on the board
                    solveRec(nextRow, nextCol, solutions); // Recursively solve for the next cell
                    board[row][col] = 0;      // Backtrack by resetting the cell back to empty (0)
                }
            }
        }
    }

public:                 // Public interface for the SudokuSolver class
    SudokuSolver(const std::vector<std::vector<int>>& initialBoard) { // Constructor taking initial
board
        board = initialBoard;               // Initialize internal board with the provided puzzle
    }

    void findSolutions(std::vector<std::vector<std::vector<int>>>& solutions) { // Public method to
trigger solving
        solveRec(0, 0, solutions);          // Start recursive backtracking from top-left cell (0,0)
```

```
    }

    void printBoard() const {                // Method to print the current state of the board
        for (int r = 0; r < 9; ++r) {        // Loop through each row of the board
            for (int c = 0; c < 9; ++c) {    // Loop through each column of the row
                if (board[r][c] == 0) {      // Check if the cell value is 0 (blank)
                    std::cout << "_ ";        // Print an underscore for blank cells
                } else {                     // If the cell contains a valid digit
                    std::cout << board[r][c] << " "; // Print the digit followed by a space
                }
            }
            std::cout << "\n";               // Print a newline at the end of each row
        }
    }
};

class PuzzleManager {  // Class to manage reading files and running the solver
public:                // Public methods for file management
    void run() {       // Main execution method to process all 5 puzzle files
        for (int fileIndex = 1; fileIndex <= 5; ++fileIndex) { // Loop from 1 to 5 for puzzle files
            std::string filename = "puzzle" + std::to_string(fileIndex) + ".txt"; // Construct file name
string
            std::ifstream infile(filename);  // Open the file input stream

            std::cout << "File: " << filename << "\n"; // Print the current file name

            if (!infile.is_open()) {         // Check if the file failed to open
                std::cout << "Error: Could not open file " << filename << "\n\n"; // Print error message
                continue;                    // Skip to the next file iteration
            }

            std::vector<std::vector<int>> initialBoard(9, std::vector<int>(9, 0)); // Initialize 9x9 grid
with zeros
            bool readSuccess = true;         // Flag to track successful file parsing
            for (int r = 0; r < 9; ++r) {    // Loop through 9 rows of the puzzle file
                for (int c = 0; c < 9; ++c) { // Loop through 9 columns per row
                    char ch;                 // Variable to hold the character read from file
                    if (!(infile >> ch)) {   // Attempt to read a character from the file stream
                        readSuccess = false;  // Mark read failure if insufficient data exists
                        break;               // Break out of column loop
                    }
                    if (ch == '_') {         // Check if character represents a blank cell
                        initialBoard[r][c] = 0; // Store 0 for blank cells internally
                    } else if (ch >= '1' && ch <= '9') { // Check if character is a digit 1 through 9
```

```
                initialBoard[r][c] = ch - '0'; // Convert char digit to integer and store
            } else {                // Handle unexpected characters in the file
                initialBoard[r][c] = 0; // Default unexpected characters to blank
            }
        }
        if (!readSuccess) break;      // Break outer loop if reading failed
    }
    infile.close();                 // Close the file stream safely

    std::cout << "Original Puzzle:\n"; // Print header for original puzzle view
    SudokuSolver tempPrinter(initialBoard); // Create temporary solver instance just for
printing original
    tempPrinter.printBoard();         // Print the original puzzle grid

    std::vector<std::vector<std::vector<int>>> solutions; // Vector to store all discovered
solutions
    SudokuSolver solver(initialBoard); // Create main solver instance with initial puzzle board
    solver.findSolutions(solutions);  // Execute solver to populate all solutions

    if (solutions.empty()) {          // Check if no solutions were found by the solver
        std::cout << "No solution found\n\n"; // Print message indicating no solution exists
    } else {                          // If one or more solutions were found
        std::cout << "Solutions found (" << solutions.size() << "):\n"; // Print total solution count
        for (size_t s = 0; s < solutions.size(); ++s) { // Loop through each found solution
            std::cout << "Solution " << (s + 1) << ":\n"; // Print solution index header
            SudokuSolver solPrinter(solutions[s]); // Create solver instance for current solution
grid
            solPrinter.printBoard();  // Print the solution board grid
            std::cout << "\n";        // Print spacing newline between solutions
        }
    }
  }
}
};


int main() {          // Main entry point of the C++ program
  PuzzleManager manager; // Create an instance of the PuzzleManager class
  manager.run();     // Invoke the run method to process all puzzle files
  return 0;          // Return 0 to indicate successful program termination
}
```

- Code Analysis:

e. Comparison
Correctness:
- Both codes passed all 5 puzzles with identical results for each.
- Edge cases and error handling:
    - Input: random bad character on a file
        - ChatGPT: Prints an error and skips the file
        - Gemini: Turns the x into a blank and solves a different puzzle which could break the program but is a simple fix.
    - File of only 3 rows:
        - ChatGPT: Prints error correctly.
        - Gemini: No output; process was killed.
    - Two 5's in one row:
        - Both AI's report a false solution that can be fixed within the code changes though.

Time Complexity:
- O(9^k) for both GenAIs as they both use recursion with up to 9 branches on each cell.
- isValid is a function inside both and runs for all 27 cells so it is O(1)
- ChatGPT's findEmptyCell function restarts at (0,0) each recursive call and this leads to up to 81 extra checks each step while Gemini goes by a (row, col) to skip re going over covered boxes
- Empirical:
    - 0.158s for the final program
    - ChatGPT: 0.175s
    - Gemini: 0.150s

Space Complexity:
- The board for both is a fixed size and the stack only goes 81 frames deep no matter what
- ChatGPT: Prints each solution as it's found so it doesn't take extra space
- Gemini: Stores every solution in a vector, so the space grows with every solution the space then is O(S x 81)

Maintainability:
- Both codes commented on every line
- ChatGPT created only one class to do everything. Gemini uses two classes but has a long run() function that handles printing, reading, and solving.
- Both use clear names
- ChatGPT is stronger with error handling as seen in correctness

f. Code Chosen:

I picked Gemini's code to go off of its two classes and the lesser time complexity by moving the recursive call with a cursor. It has weaknesses with its error handling and storing of solutions but those can be adjusted.

g. Improvements:

1. Correctness:
    a. Changed line inside run() which loops through to create the vector matrix from the file in order to detect failed characters and represent that as a failure instead of how Gemini did it originally which was by setting it to a blank.
    b. Created a givens are valid function to detect if a digit is repeated in a row, column, or 3x3 grid because Gemini had failed to detect that before. And print that there is no solution found.
2. Storage:
    a. Edited the solveRec function to print the solution after it is found instead of storing it to print at the end. This saves storage by deleting the need to save multiple solutions per file.
3. Maintainability:
    a. Shortened run into a separate function to make it easier to maintain.
    b. Deleted tempPrinter as it was redundant to build two solvers when we can make one print it while it’s not solved then solve it.